

SPACE INVADERS

Documentación Técnica de Arquitectura

ARMv7 – CPULATOR

Autores:

Juan Sebastián Puerto

Juan Diego Forero

Juan David Lizcano

Profesor:

Edson Fabian Mojica Rodríguez

Universidad Industrial de Santander - UIS

Arquitectura de Computadores – grupo E1

2026-1

TABLA DE CONTENIDOS

1. Introducción
2. Descripción General del Sistema
3. Arquitectura del Software
4. Organización del Código Assembly
5. Sistema de Audio
6. Sistema Gráfico y UI
7. Mecánica del Juego
8. Game Loop Principal
9. Gestión de Estados del Juego
10. Gestión de Memoria y Registros ARM
11. Memory-Mapped I/O y Hardware
12. Decisiones de Diseño de Ingeniería
13. Resultados y Validación
14. Mejoras Futuras
15. Conclusiones
16. Anexos Técnicos

1. INTRODUCCIÓN

Este documento presenta la documentación técnica completa de una implementación del clásico videojuego Space Invaders en lenguaje Assembly ARMv7, ejecutándose sobre la plataforma DE1-SoC en el emulador CPUlator. Este proyecto representa una exploración profunda de programación a nivel de hardware, arquitectura de sistemas embebidos, y diseño de software en tiempo real.

El objetivo fundamental del proyecto es demostrar la capacidad de implementar un sistema interactivo complejo utilizando exclusivamente instrucciones nativas de la arquitectura Cortex-A9, incluyendo gestión de memoria, entrada/salida de hardware, manejo de interrupciones, y renderizado gráfico a nivel de píxel.

1.1 Contexto Técnico

La plataforma DE1-SoC es un sistema integrado que combina un procesador ARM Cortex-A9 con memoria, periféricos de entrada/salida (teclado, pantalla), y controladores de audio. El desarrollo en Assembly requiere comprensión profunda de:

- Arquitectura ARMv7: set de instrucciones, modos de procesador, gestión de interrupciones
- Memory-Mapped I/O: interacción con periféricos a través de direcciones de memoria
- Programación de tiempo real: sincronización de frames, manejo de eventos asincronos
- Algoritmos de colisión: detección de impactos entre múltiples objetos dinámicos

1.2 Versión del Proyecto

Este documento describe la versión 5.0 del proyecto, que incorpora avances significativos sobre versiones anteriores:

- Barreras tipo 'bunker' con 3 niveles de daño (pixelados a 4x4 píxeles)
- Sistema de disparos enemigos: hasta 4 balas simultáneas con lógica de selección de objetivo

- Sistema de niveles (1-3) con progresión de dificultad
- Sistema de vidas y Game Over
- Animaciones de explosión sincronizadas con timer del sistema
- Integración completa de audio con BGM y SFX

2. DESCRIPCIÓN GENERAL DEL SISTEMA

2.1 Especificaciones de Hardware

Componente	Especificación	Dirección Base (si aplica)
Procesador	ARM Cortex-A9 (ARMv7)	N/A
Memoria RAM	DDR3 512 MB	0x00000000
Pantalla	320×240, 16-bit RGB565	0xC8000000 (PIXBUF)
Teclado PS/2	Entrada de jugador	0xFF200050 (KEYS_BASE)
PS/2 Controller	Mouse/Teclado	0xFF200100 (PS2_BASE)
Timer de Intervalo	Sincronización 50 FPS	0xFF202000 (TIMER_BASE)
Controlador de Audio	Audio Core (DAC)	0xFF203040 (AUDIO_BASE)
LEDs Rojos	Display de estado	0xFF200000 (LEDR_BASE)
Display HEX	7-segment, 4 dígitos	0xFF200020 (HEX_BASE)

2.2 Arquitectura de Alto Nivel

El sistema sigue una arquitectura de máquina de estados con un game loop síncrono impulsado por interrupciones de timer. La organización se puede resumir en:

- STATE_SPLASH: Pantalla inicial, espera entrada de usuario
- STATE_PLAYING: Loop de juego principal, actualización y renderizado 50 veces por segundo
- STATE_GAME_OVER: Pantalla de derrota, espera reinicio

- STATE_VICTORY: Pantalla de victoria intermedia, avance de nivel

Cada frame (20 ms a 50 FPS) ejecuta en este orden: validación de colisiones, actualización de posiciones, borrado de gráficos antiguos, cálculo de nuevas posiciones, renderizado.

3. ARQUITECTURA DEL SOFTWARE

3.1 Estructura Modular

El código Assembly se organiza en módulos funcionales lógicos, cada uno responsable de un aspecto del sistema:

Módulo	Responsabilidad	Funciones Clave
Inicialización	Setup de hardware, IRQs, periféricos	SET_IRQs, CONFIG_INTERVAL_TIMER, AUDIO_INIT
Game Loop	Coordinación de frame	MainLoop, WaitFrame, UpdateExplosions
Entrada	Lectura de teclado PS/2	ReadKeys, MoveCannon
Física	Movimiento de objetos	MoveBullet, MoveInvaders, MoveEBullet
Colisiones	Detección de impactos	CheckBulletVsInvaders, CheckEBulletVsCannon
Gráficos	Renderizado 2D	DrawAll, DrawInvaders, DrawBarriers, WritePixelSafe
Audio	Reproducción BGM/SFX	BGM_Play, Audio_SetVolume, AUDIO_INIT
UI	Información en pantalla	DrawUI, UpdateHexDisplay, DrawLevelIndicator
Barreras	Sistema defensivo	DrawBarriers, DamageBarrierAt

Disparos Enemigos	IA de ataque	MaybeEnemyShoot, MoveEBullet, CheckEBulletVsCannon
-------------------	--------------	--

3.2 Flujo de Control Principal

El programa sigue este flujo general:

```
_start → SET_IRQs → CONFIG_PERIFÉRICOS → STATE_SPLASH  ↓ splash_loop (espera
reset_flag)  ↓ MainLoop:  ├── WaitFrame (sincronización timer)  ├── Colisiones:
CheckBulletVsInvaders, CheckEBulletVsCannon, etc.  ├── Movimiento: MoveCannon,
MoveBullet, MoveInvaders, MaybeEnemyShoot  ├── Borrado: EraseAll  ├──
Renderizado: DrawAll, DrawBarriers, DrawUI  └─ Validación: CheckAllDead,
CheckGameOver, CheckVictory
```

4. ORGANIZACIÓN DEL CÓDIGO ASSEMBLY (.data y .text)

4.1 Sección .data (Variables Globales)

La sección .data contiene todas las variables globales del sistema, organizadas por categoría:

Constantes y Configuración:

```
WIDTH = 320 px    | Ancho pantalla (resolución) HEIGHT = 240 px    | Alto pantalla
PIXBUF = 0xC8000000 | Base de framebuffer (Memory-Mapped I/O) INV_COLS = 11
| Columnas de invasores en grid INV_ROWS = 5    | Filas de invasores en grid
INV_COUNT = 55    | Total invasores (11×5) CANNON_Y = 220 px    | Posición Y del
cañón (cerca del borde)
```

Arreglos Dinámicos (en RAM):

Variable	Tipo	Tamaño	Propósito
inv_alive	array 32-bit	55 × 4 bytes	Estado de cada invasor (0=muerto, 1=vivo, 2=explotando)
inv_expl_timer	array 32-bit	55 × 4 bytes	Timer de explosión para cada invasor

barrier_blocks	array 32-bit	72 × 4 bytes	Estado daño de cada bloque (3=intacto, 0=destruido)
bullet_x, bullet_y	32-bit	4 bytes c/u	Posición bala jugador
ebullet_x[4]	32-bit × 4	16 bytes	Posiciones X de 4 balas enemigas
cannon_x	32-bit	4 bytes	Posición X del cañón

4.2 Sección .text (Código Ejecutable)

El código se organiza en secciones lógicas secuenciales:

- Vector Table: Tabla de vectores de excepciones (offset 0x00-0x1C)
- _start: Inicialización de stack, modo de procesador, periféricos
- MainLoop: Core del juego, manejo de estados
- Funciones de Entrada: ReadKeys, MoveCannon
- Funciones de Física: MoveBullet, MoveInvaders, MoveEBullet
- Funciones de Colisión: CheckBulletVsInvaders, DamageBarrierAt, etc.
- Funciones de Gráficos: WritePixelSafe, DrawInvaders, DrawBarriers
- Funciones de Audio: BGM_Play, AUDIO_INIT

5. SISTEMA DE AUDIO

5.1 Arquitectura del Audio Core

El Audio Core de la DE1-SoC es un controlador de audio memory-mapped a 0xFF203040. El sistema de audio implementa la reproducción de múltiples pistas de sonido mediante una arquitectura de switches de contexto.

Sonido ID	Identificador	Tipo	Uso en Juego
0	SND_NONE	Silencio	Pausar música

1	SND_SPLASH	BGM (Theme Clásico)	Pantalla inicial
2	SND_THEME	BGM (Tema Juego)	Música principal
3	SND_GAMEOVER	SFX	Pantalla derrota
4	SND_DEATH	SFX	Muerte jugador
5	SND_BULLET	SFX	Disparo del jugador

5.2 Implementación de BGM_Play

La función BGM_Play es la interfaz de software para controlar el audio core:

```
BGM_Play: PUSH {R0, LR} LDR R1, =AUDIO_BASE    @ R1 = 0xFF203040 STR R0,
[R1]          @ Escribir ID de sonido en registro de control POP {R0, LR} BX LR
```

El Audio Core interpreta automáticamente el ID de sonido y activa la pista correspondiente. Este mecanismo permite cambios dinámicos de música sin interrupciones de audio.

5.3 Cambios de Estado Sonoro

El sistema transiciona entre estados sonoros en momentos clave:

Evento	Música Anterior	Música Nueva	Línea de Código
Inicio de splash	N/A	SND_SPLASH	BL BGM_Play @ _start
Inicio juego	SND_SPLASH	SND_THEME	BL BGM_Play @ InitGame
Game Over	SND_THEME	SND_GAMEOVER	BL BGM_Play @ GameOver_Loop
Victoria nivel	SND_THEME	SND_THEME	Sin cambio (continúa)
Victoria final	SND_THEME	SND_SPLASH	BL BGM_Play @ VL_FinalVictory

5.4 Sincronización de Audio con Game Loop

El Audio Core opera de forma asincrónica e independiente del game loop. Los cambios de pista se aplican inmediatamente al escribir en 0xFF203040, sin requerir espera o confirmación. Esto permite que la música continúe reproduciéndose mientras el procesador ejecuta la lógica del juego.

6. SISTEMA GRÁFICO Y UI

6.1 Framebuffer Memory-Mapped

La pantalla DE1-SoC es controlada por un framebuffer memory-mapped a 0xC8000000. Cada píxel se representa en formato RGB565 (16 bits), permitiendo 65,536 colores.

Resolución: 320×240 píxeles Formato: 16-bit RGB565 - R: bits 15-11 (5 bits, valores 0-31) - G: bits 10-5 (6 bits, valores 0-63) - B: bits 4-0 (5 bits, valores 0-31) Stride: 512 píxeles por fila (1024 bytes) Offset de píxel (x,y): (y × 1024) + (x × 2) bytes

6.2 Paleta de Colores

Color	RGB565 Hex	Uso en Juego
Negro	0x0000	Fondo, píxeles borrados
Blanco	0xFFFF	Texto UI
Rojo	0xF800	Invasores, barrera dañada
Verde	0x07E0	Barrera intacta, indicadores
Azul	0x001F	Cañón, UI
Cyan	0x07FF	Balas jugador (verde+azul)
Magenta	0xF81F	Balas enemigos
Amarillo	0xFFE0	UI especial, splash
Gris	0x8410	Áreas neutras

6.3 Funciones de Renderizado

El sistema de gráficos implementa funciones de bajo nivel para actualizar la pantalla eficientemente:

Función	Parámetros	Descripción
WritePixelSafe	R0=x, R1=y, R2=color	Escribe píxel si está dentro de pantalla
ClearScreen	Ninguno	Llena pantalla de negro
DrawRect	R0=x, R1=y, R2=w, R3=h, R4=color	Dibuja rectángulo relleno
DrawInvader	R0=idx	Dibuja invasor individual con forma específica
DrawCannon	Ninguno	Dibuja cañón en posición actual
DrawBullet	Ninguno	Dibuja bala activa del jugador
DrawEBullet	Ninguno	Dibuja bala enemiga
DrawBarriers	Ninguno	Dibuja todas las barreras con estado actual

6.4 UI En Pantalla

La interfaz de usuario muestra información crítica del juego:

- LEVEL X: Número de nivel en esquina superior izquierda (dígitos pixelados 5×5)
- Score: Puntuación en display HEX de 7 segmentos (0-9999)
- Vidas: Número de vidas restantes (max 3)

El sistema DrawUI se ejecuta cada frame para mantener información actualizada. La puntuación se calcula sumando 10 puntos por cada invasor destruido.

7. MECÁNICA DEL JUEGO

7.1 Jugador (Cañón)

El cañón es controlado por el jugador usando las teclas del teclado. Su posición X es variable, Y es fija en 220 píxeles (cerca del borde inferior).

Propiedad	Valor	Descripción
Posición Y	220 px	Fija, cercana a borde inferior
Ancho	~8 px	En sprites dibujados
Altura	~10 px	En sprites dibujados
Rango X	0-301 px	Movimiento permitido
Velocidad	5 px/frame	Paso de movimiento
Control	Teclas ← →	PS/2 keyboard input
Invencibilidad	60 frames	Después de ser golpeado

La función MoveCannon lee el estado del teclado y actualiza cannon_x. El clamping se aplica automáticamente para mantener el cañón dentro de los límites de pantalla.

7.2 Enemigos (Invasores)

Los invasores forman una matriz de 11×5 (55 enemigos total) que se desplaza horizontalmente y hacia abajo de forma coordinada.

Propiedad	Valor	Descripción
Grid Inicial	11 cols × 5 filas	55 invasores totales
Espaciado X	22 px	Distancia horizontal entre invasores
Espaciado Y	18 px	Distancia vertical entre invasores
Movimiento X	2 px/frame	Paso de desplazamiento horizontal
Movimiento Y	6 px/frame	Paso hacia abajo al cambiar dirección
Frecuencia Movimiento	20 frames	Cada 20 frames se mueven
Velocidad Inicial	INV_MOVE_COUNTER=0	Sin delay inicial

Los invasores se mueven como un bloque cohesivo. El vector inv_offset_x e inv_offset_y mantienen el desplazamiento global, permitiendo que todos se muevan de forma coordinada sin actualizar 55 variables individuales.

7.3 Balas del Jugador

El jugador puede disparar una bala a la vez. Las balas se mueven hacia arriba a velocidad constante.

Propiedad	Valor	Descripción
Estado Inactivo	0xFFFE	Valor especial para bala inactiva
Velocidad	8 px/frame	Desplazamiento hacia arriba
Cooldown	2 frames	Mínima espera entre disparos
Origen	Cañón X	La bala surge de la posición del cañón
Color	0x07FF (Cyan)	Verde+Azul en RGB565
Colisión	Invasores, Barreras	Se desactiva al impactar

El sistema de balas implementa un mecanismo de 'un disparo activo' simultáneamente. Cuando el jugador presiona disparo, si no hay bala activa, se crea una nueva en la posición del cañón. La variable `bullet_x` se actualiza cada frame en `MoveBullet()`.

7.4 Balas Enemigas

Los invasores disparan de forma aleatoria (IA) hacia el cañón. Hay soporte para hasta 4 balas enemigas simultáneas.

Propiedad	Valor	Descripción
Cantidad Simultánea	4 balas	<code>ebullet[1-4]_x/y</code>
Estado Inactivo	0xFFFD	Valor para bala inactiva
Velocidad	1 px/frame	Desplazamiento hacia abajo
Color	0xF81F (Magenta)	Rojo+Azul en RGB565
Frecuencia Disparo	40-90 frames	Random entre <code>ESHOOT_MIN</code> y <code>+ESHOOT_RANGE</code>

Selección Objetivo	Random	Invasor aleatorio elige disparar
Colisión	Jugador, Barreras	Daña jugador (-1 vida) o degrada barrera

La función MaybeEnemyShoot implementa la lógica de IA. Cada frame, si eshoot_timer <= 0, selecciona un invasor vivo aleatorio y activa una bala enemiga en su posición. El timer se reinicia a valor random en rango.

7.5 Barreras (Bunkers)

Hay 3 barreras posicionadas entre los invasores y el cañón. Cada barrera consta de 6×4=24 bloques de 4×4 píxeles.

Propiedad	Valor	Descripción
Cantidad	3 barreras	Posiciones X fijas: 50, 148, 246
Bloques por Barrera	24 bloques	6 cols × 4 filas
Tamaño Bloque	4×4 píxeles	Unidad de daño
Posición Y	195 px	Altura fija
Estados Bloque	3=intacto, 2=dañado1, 1=dañado2, 0=destruido	4 niveles
Color Intacto	0x07E0 (Verde)	BLOCK_FULL
Color Dañado1	0x03E0 (Verde med)	BLOCK_STATE_2
Color Dañado2	0x0160 (Verde oscuro)	BLOCK_STATE_1
Impacto	Bala jugador u enemiga	Reduce estado en 1

Las barreras proporcionan cobertura defensiva temporal. Tanto las balas del jugador como las enemigas las degradan. El sistema DamageBarrierAt() busca la barrera impactada, localiza el bloque específico, y reduce su estado.

7.6 Sistema de Colisiones

El sistema de colisiones se ejecuta al inicio de cada frame, validando todas las combinaciones de objetos:

Tipo Colisión	Función	Acción
Bala Jugador vs Invasor	CheckBulletVsInvaders	Invasor → explotando (+score), bala → inactiva
Bala Jugador vs Barrera	CheckBulletVsBarriers	Bloque → dañado, bala → inactiva
Bala Enemiga vs Barrera	CheckEBulletVsBarriers	Bloque → dañado, bala → inactiva
Bala Enemiga vs Cañón	CheckEBulletVsCannon	Jugador → herido (-1 vida), bala → inactiva
Invasor vs Cañón (contacto)	CheckCannonCollision	Game Over inmediato
Invasores llegan a fondo	CheckInvadersReachedBottom	Game Over inmediato

8. GAME LOOP PRINCIPAL

8.1 Sincronización con Timer

El game loop es impulsado por interrupciones de timer configurado para generar una interrupción cada 20 ms (50 FPS).

```
CONFIG_INTERVAL_TIMER: LDR R0, =TIMER_BASE LDR R1, =0x0000004B @  
Configurar para 20ms STR R1, [R0] LDR R1, =0x00000004 @ Habilitar interrupciones  
STR R1, [R0, #8]
```

Cuando ocurre la interrupción, el controlador IRQ establece `frame_ready = 1`. El game loop espera este flag antes de proceder.

8.2 Orden de Operaciones por Frame

La ejecución de cada frame sigue un orden cuidadosamente diseñado para evitar inconsistencias:

- 1. Sincronización:** WaitFrame espera `frame_ready = 1`
- 2. Validación de Colisiones:** Detecta todos los impactos. Este paso debe ser primero para evitar que objetos 'penetren' las barreras.
- 3. Actualización de Explosiones:** Decrementa timers de animación, marca invasores como muertos cuando termina la explosión.
- 4. Borrado de Gráficos Antiguos:** EraseAll redibuja en negro las posiciones anteriores usando `prev_cannon_x`, `prev_bullet_x`, etc.
- 5. Actualización de Física:** MoveCannon, MoveBullet, MoveInvaders, MoveEBullet recalculan posiciones.
- 6. Lógica de Enemigos:** MaybeEnemyShoot actualiza IA, decide si disparar, selecciona objetivo aleatorio.
- 7. Renderizado de Nuevo Estado:** DrawAll dibuja cañón, balas, invasores en nuevas posiciones. DrawBarriers renderiza barreras con estado actual.
- 8. UI:** DrawUI actualiza puntuación, nivel, vidas.
- 9. Validación de Fin de Juego:** CheckAllDead (victoria), CheckGameOver (derrota).

8.3 Estructura de Código MainLoop

```
MainLoop: LDR R0, =game_state LDR R0, [R0] CMP R0, #STATE_SPLASH BEQ
Splash_Loop CMP R0, #STATE_GAME_OVER BEQ GameOver_Loop WaitFrame: LDR
R0, =frame_ready LDR R1, [R0] CMP R1, #0 BEQ WaitFrame @ Bloquear hasta
frame_ready MOV R1, #0 STR R1, [R0] @ Resetear flag @ Colisiones →
Borrado → Movimiento → Renderizado BL CheckBulletVsInvaders BL
CheckEBulletVsCannon ... BL UpdateExplosions BL EraseAll BL MoveCannon BL
```

```
MoveInvaders BL DrawAll BL DrawBarriers BL DrawUI BL CheckAllDead B
MainLoop
```

9. GESTIÓN DE ESTADOS DEL JUEGO

9.1 Máquina de Estados

El juego implementa una máquina de estados de 4 estados principales:

Estado	ID	Descripción	Transiciones
SPLASH	3	Pantalla inicial + menú	→ PLAYING (reset_flag=1)
PLAYING	0	Loop de juego activo	→ GAME_OVER (vidas=0) o VICTORY (todos muertos)
GAME_OVER	1	Pantalla derrota	→ SPLASH (reset_flag=1)
VICTORY	2	Nivel completado	→ PLAYING (nivel<MAX) o SPLASH (nivel=MAX)

9.2 Variables de Control

El estado global se controla mediante estas variables críticas:

Variable	Rango	Propósito
game_state	0-3	Estado actual (PLAYING, GAME_OVER, VICTORY, SPLASH)
reset_flag	0-1	Flag de entrada para transiciones de estado
current_level	1-MAX_LEVEL	Nivel actual (1, 2, 3)
lives	0-3	Vidas restantes del jugador
score	0-9999	Puntuación acumulada
frame_ready	0-1	Flag de sincronización de timer
invincible_frames	0-60	Frames de invencibilidad post-daño

9.3 Flujo SPLASH → PLAYING → GAME_OVER → SPLASH

El flujo completo de un ciclo de juego es:

SPLASH: - Mostrar pantalla inicial - Reproducir SND_SPLASH (tema clásico) - Esperar reset_flag = 1 (cualquier tecla) - Ir a InitGame
InitGame: - Resetear vidas = 3, score = 0
- Setear game_state = PLAYING - Reproducir SND_THEME - Inicializar invasores todos vivos
PLAYING (MainLoop): - Cada frame: colisiones, movimiento, renderizado - Si CheckAllDead: game_state = VICTORY - Si CheckGameOver: game_state = GAME_OVER
VICTORY: - Mostrar pantalla de victoria - Si nivel < MAX: incrementar nivel, volver a InitGame - Si nivel = MAX: mostrar winner screen, volver a SPLASH
GAME_OVER: - Mostrar pantalla de derrota - Reproducir SND_GAMEOVER - Esperar reset_flag = 1 - Resetear nivel = 1, vidas = 3, ir a SPLASH

10. GESTIÓN DE MEMORIA Y REGISTROS ARM

10.1 Uso de Registros ARM

La arquitectura ARMv7 proporciona 16 registros de propósito general (R0-R15). El proyecto asigna cada registro con propósito específico:

Registro	Convención	Uso Principal
R0	Argument	Parámetro 1, valor de retorno
R1	Argument	Parámetro 2
R2	Argument	Parámetro 3
R3	Argument	Parámetro 4
R4-R11	Variable	Variables locales (se preservan)
R12 (IP)	Intra-procedure	Temporal, perdible entre llamadas
R13 (SP)	Stack Pointer	Pila (crecimiento descendente)
R14 (LR)	Link Register	Dirección retorno subrutina
R15 (PC)	Program Counter	Contador de programa (no asignable directamente)

10.2 Stack y Convención de Llamadas

Las funciones siguen la convención ARM EABI (Embedded ABI):

- Argumentos: R0-R3 (máximo 4 argumentos sin stack)
- Retorno: R0 (o R0:R1 para 64-bit)
- Preservación: Funciones deben preservar R4-R11, SP
- Stack: PUSH/POP se usan para guardar registros al inicio/final de función

Ejemplo: MiFunc: PUSH {R4-R7, LR} @ Guardar registros preservados + retorno @ ...
código usando R4-R7 ... POP {R4-R7, PC} @ Restaurar y retornar

10.3 Organización de Memoria

Región	Rango	Contenido
Vector Table	0x00000000-0x0000001C	Tabla de excepciones
Código (.text)	0x00000000+	Instrucciones Assembly
Datos (.data)	Después de .text	Variables globales, tablas
Heap	Dinámico (arriba)	Asignación dinámica (no usado aquí)
Stack	0x800000 (descend.)	Pila de llamadas
Periféricos I/O	0xFF200000+	Memory-mapped I/O

10.4 Optimización de Memoria

El código usa varias técnicas para minimizar uso de memoria:

- inv_offset_x/y: En lugar de 55 variables individuales, se mantienen desplazamientos globales
- Índices de array: Se usan R1, LSL #2 para acceder directamente a arrays mediante offset de registro
- Zero-initialization: inv_alive[55] se inicializa en loop, no elemento a elemento

11. INTERACCIÓN HARDWARE–SOFTWARE (MEMORY-MAPPED I/O)

11.1 Periféricos Memory-Mapped

La plataforma DE1-SoC expone todos los periféricos como direcciones de memoria. El sistema interactúa directamente con hardware sin drivers complejos:

Periférico	Dirección Base	Función	Tipo I/O
Pixel Buffer	0xC8000000	Framebuffer gráfico	Write-only
Timer de Intervalo	0xFF202000	Generador de interrupciones	Read/Write
Teclado PS/2	0xFF200050	Entrada de usuario	Read
PS/2 Controller	0xFF200100	Control de periféricos PS/2	Write
Display HEX	0xFF200020	7 dígitos segmentos	Write
LEDs Rojos	0xFF200000	LEDs de estado	Write
Audio Core	0xFF203040	Controlador de audio	Write

11.2 Acceso a Framebuffer

El framebuffer de 320×240 píxeles en RGB565 se accede como un array lineal de 16-bit words:

Cálculo de offset: $\text{Offset (bytes)} = (Y \times 1024) + (X \times 2)$ Ejemplo: Píxel en (100, 50)
 $\text{Offset} = (50 \times 1024) + (100 \times 2) = 51200 + 200 = 51400 \text{ bytes}$ Dirección =
 $0xC8000000 + 51400$ En ARM Assembly (usando shift): `LSL R7, R5, #10` @ $R7 = Y \times 1024$ (shift izquierda 10 bits) `ADD R7, R1, R7` @ $R7 = \text{base} + Y \times 1024$ `ADD R7, R7, R4, LSL #1` @ $R7 += X \times 2$ (shift izquierda 1 bit)

11.3 Lectura de Entrada de Teclado

El teclado PS/2 expone su estado a través de 0xFF200050. La lectura se realiza de forma polling:

ReadKeys: `LDR R0, =KEYS_BASE` @ $R0 = 0xFF200050$ `LDR R1, [R0]` @ Leer registro de estado @ $R1[\text{bit } 0] = \text{izquierda}$, $R1[\text{bit } 1] = \text{derecha}$, etc. `AND R1, R1, #0x3` @ Enmascarar bits relevantes `BX LR`

11.4 Control de Audio

El Audio Core se controla escribiendo el ID de sonido en su registro de control:

```
BGM_Play (R0 = ID_sonido): LDR R1, =AUDIO_BASE @ R1 = 0xFF203040 STR R0,
[R1] @ Escribir ID en control BX LR @ El Audio Core maneja el resto
```

11.5 Display HEX 7-Segmentos

El display HEX de 4 dígitos a 0xFF200020 se actualiza escribiendo un 32-bit word con codificación especial:

```
Formato: Bits 31-24: Dígito 3 (7-segment encoding) Bits 23-16: Dígito 2 Bits 15-8:
Dígito 1 Bits 7-0: Dígito 0 Codificación 7-segment: Bit 6 (MSB) a Bit 0 mapean a
segmentos a-g del display Ejemplo: Dígito '0' = 0x3F (0111111 binario)
```

12. DECISIONES DE DISEÑO DE INGENIERÍA

12.1 Diseño de Sistema de Explosiones

Versión v5 introduce un sistema de explosión visual mediante animación de sprites de corta duración. En lugar de eliminar invasores instantáneamente, se marcan como explotando (`inv_alive[i] = 2`) con un timer.

```
Estados de invasor: 0 = DEAD (destruido, no renderizar) 1 = ALIVE (activo, renderizar
sprite normal) 2 = EXPLODING (en explosión, renderizar sprite alternativo)
```

La función `UpdateExplosions()` decrementa `inv_expl_timer[i]` cada frame. Cuando llega a 0, `inv_alive[i] → 0` (DEAD). Esto proporciona feedback visual sin complejidad de animación.

12.2 Selección Aleatoria de Invasor Disparador

`MaybeEnemyShoot` selecciona un invasor aleatorio para disparar. El generador RNG simple usa una fórmula lineal congruencial:

```
Simple RNG: rng_seed = (rng_seed * 37 + 1) mod 2^32 return (rng_seed / 256) mod
num_invasores Esta no es criptografía, pero es suficiente para aleatoriedad de juego.
```

El algoritmo evita bias iterando sobre invasores vivos hasta encontrar uno válido para disparar.

12.3 Sistema de Barreras Pixeladas

Las barreras se implementan como arrays 2D de 'bloques', no como gráficos continuos. Cada bloque es un píxel de 4×4. Esto permite:

- Daño visual: bloques degradan estado en lugar de desaparecer instantáneamente
- Colisión granular: balas impactan a nivel de bloque individual, no barrera completa
- Eficiencia de memoria: 72 integers (24 bloques × 3 barreras) en lugar de sprites continuos

12.4 Sincronización de Frame 50 FPS

El timer del sistema se configura para generar una interrupción cada 20 ms (50 FPS). Esta frecuencia se elige porque:

- Suficientemente suave: El ojo humano percibe ~30 FPS como fluido; 50 es cómodo
- Período exacto: 20 ms es potencia de 2 en configuración de timer, simplifica aritmética
- Carga moderada: A 50 FPS, la CPU tiene ~2 ms de tiempo para toda la lógica + renderizado

12.5 Gestión de Invencibilidad Post-Daño

Cuando el jugador es golpeado por una bala enemiga, se le otorgan 60 frames de invencibilidad. Durante estos frames, las balas enemigas adicionales no lo pueden dañar. Esto evita situaciones 'unfair' donde múltiples balas lo eliminan instantáneamente.

El contador `invincible_frames` se decrementa en `MoveCannon()`. Cuando llega a 0, la invencibilidad se pierde.

13. RESULTADOS DEL SISTEMA

13.1 Validación Funcional

El sistema ha sido validado en el emulador CPULATOR bajo la siguiente configuración:

Aspecto	Resultado
---------	-----------

Renderizado	✓ Gráficos fluidos a 50 FPS sin parpadeo
Entrada de usuario	✓ Teclado PS/2 responde inmediatamente
Física	✓ Movimiento de invasores sincronizado y predecible
Colisiones	✓ Todas las combinaciones (bala-invasor, bala-barrera, etc.) detectadas correctamente
Audio	✓ BGM y SFX reproducen sin interrupciones
Estados	✓ Transiciones SPLASH → PLAYING → GAME_OVER funcionan
Niveles	✓ Progresión de dificultad (1-3) implementada
Barreras	✓ Degradación visual de bloques bajo fuego

13.2 Métricas de Performance

Métrica	Valor	Análisis
FPS observado	50 (sincronizado)	Perfecto, sin frame drops observados en CPULator
Tiempo de frame	20 ms	Exacto según configuración de timer
Uso de memoria (.text)	~60 KB	Eficiente para arquitectura embebida
Uso de memoria (.data)	~8 KB	Variables + tablas para invasores, balas
Latencia entrada→salida	<1 frame	Entrada se refleja en pantalla dentro de 20 ms
Ocupación CPU	~15%	Estimado; suficiente headroom para extensiones

13.3 Casos de Uso Validados

- Usuario mueve cañón ← → : Posición se actualiza fluidamente sin lag

- Usuario dispara múltiples veces rápidamente: Solo 1 bala activa, disparos posteriores se ignoran hasta cooldown
- Invasores avanzan hacia cañón: Detección de contacto causa Game Over
- Invasor es destruido: Sprite de explosión es visible, puntuación se incrementa
- Barrera recibe fuego: Bloques se degradan visualmente, eventualmente desaparecen
- Bala enemiga impacta cañón: Vidas decremantan, cañón recibe invencibilidad temporal

14. MEJORAS FUTURAS

14.1 Extensiones Gráficas

- Sprites animados: Invasores con 2-3 frames de animación para simular movimiento de 'patas'
- Pantalla de puntuación final: Mostrar high scores persistentes (requiere almacenamiento en EEPROM)
- Efectos visuales: Trails de balas, partículas de explosión

14.2 Mejoras de Gameplay

- Niveles adicionales (4+): Aumentar velocidad, densidad de enemigos, IA más compleja
- Power-ups: Munición rápida, escudo temporal, arma de área
- Diferentes tipos de invasores: Variantes con mayor vida o patrones de ataque especiales
- Modo multijugador: 2 cañones con barreras compartidas

14.3 Optimizaciones de Código

- Renderizado parcial: Actualizar solo rectángulos sucios en lugar de toda la pantalla
- SIMD (NEON): Usar instrucciones vectoriales ARMv7 para copias de memoria más rápidas
- Caché de lookups: Pre-computar offsets de framebuffer para filas

14.4 Características de Ingeniería

- Logging de debug: Información de estado en UART para análisis post-mortem
- Profiling de timer: Medir tiempo de cada función para identificar cuellos de botella
- Testeo de cobertura: Validar todos los caminos de código mediante casos de prueba sistématicos

15. CONCLUSIONES

Este proyecto demuestra la viabilidad técnica y práctica de implementar un sistema de videojuego complejo en lenguaje Assembly ARMv7 puro. A pesar de la ausencia de abstracción de lenguaje de alto nivel, el código logra rendimiento fluido, lógica responsiva, y una experiencia de usuario satisfactoria.

15.1 Logros Clave

- ✓ Implementación 100% en Assembly ARMv7: Demuestra dominio de arquitectura de bajo nivel
- ✓ Juego completo y funcional: Todas las mecánicas clásicas de Space Invaders implementadas
- ✓ Sistema de niveles progresivo: 3 niveles con aumento de dificultad
- ✓ Gráficos y audio sincronizados: Renderizado fluido a 50 FPS con BGM/SFX
- ✓ Lógica de IA: Disparos enemigos aleatorios pero deterministas
- ✓ Sistema de colisiones robusto: Múltiples tipos simultáneamente sin casos edge

15.2 Lecciones de Ingeniería

El desarrollo de este proyecto ha ilustrado varios principios fundamentales de ingeniería de sistemas embebidos:

- Diseño Top-Down con Implementación Bottom-Up: Comenzar con arquitectura de alto nivel, luego validar con primitivas de bajo nivel

- Sincronización es Crítica: Un game loop sin sincronización resulta en parpadeo y comportamiento impredecible
- Memory-Mapped I/O Simplifica Hardware: No requiere drivers complejos, solo acceso a dirección correcta
- Optimización de Memoria es Esencial: Arreglos indexados en lugar de variables individuales ahorran KBs
- Orden de Operaciones Importa: Colisiones antes de movimiento previene penetración; borrado antes de dibujo previene parpadeo

15.3 Potencial Académico

Este proyecto sirve como excelente estudio de caso para cursos de:

- Arquitectura Computacional: Mapeo directo de conceptos teóricos a hardware real
- Sistemas Embebidos: Programación de tiempo real, interrupciones, periféricos
- Algoritmos: Colisiones, generación aleatoria, máquinas de estado
- Programación de Bajo Nivel: Intrincacías del Assembly, convenciones de llamadas

16. ANEXOS TÉCNICOS

16.1 Tabla de Constantes Completa

Referencia rápida de todas las constantes críticas del código:

Constante	Valor	Tipo
WIDTH	320	Ancho pantalla (píxeles)
HEIGHT	240	Alto pantalla (píxeles)
INV_COUNT	55	Total de invasores
INV_COLS	11	Columnas grid
INV_ROWS	5	Filas grid

CANNON_Y	220	Posición Y cañón
BULLET_SPEED	8	Píxeles/frame bala jugador
EBULLET_SPEED	1	Píxeles/frame bala enemiga
INV_MOVE_EVERY	20	Frames entre movimiento invasores
BARRIER_COUNT	3	Número de barreras
BARRIER_BLOCKS	72	Total bloques (3×24)
TIMER_BASE	0xFF202000	Address timer de intervalo
AUDIO_BASE	0xFF203040	Address controlador audio
PIXBUF	0xC8000000	Address framebuffer

16.2 Mapa de Memoria de Variables

Ubicación aproximada de variables principales en sección .data:

Offset	Variable	Tamaño	Descripción	0x0000	game_state	4 bytes
	Estado actual	0x0004	lives	4 bytes	Vidas restantes	0x0008
4 bytes	Puntuación	0x000C	current_level	4 bytes	Nivel (1-3)	0x0010
cannon_x	4 bytes	Posición X cañón	0x0014	cannon_y	4 bytes	
Posición Y (fija 220)	0x0018	bullet_x	4 bytes	Posición X bala	0x001C	
bullet_y	4 bytes	Posición Y bala	0x0020	inv_alive[55]	220 bytes	
Estados invasores	0x010C	inv_expl_timer[55]	220 bytes	Timers explosión	0x01F8	
barrier_blocks[72]	288 bytes	Estados bloques	0x0318	ebullet_x[4]	16 bytes	
Posiciones balas enemigas ...						

16.3 Protocolo de Entrada PS/2

El teclado PS/2 comunica teclas presionadas a través de 0xFF200050. Cada lectura retorna un 32-bit register donde bits específicos indican estado de teclas:

Bits del registro KEYS_BASE (0xFF200050): [0] = Tecla izquierda (←) [1] = Tecla derecha (→) [2] = Tecla espacio (DISPARO) [3] = Tecla Enter (SELECT) [otros] = Reservados

16.4 Configuración de Timer para 50 FPS

El timer debe programarse para generar una interrupción cada 20 ms (período base de 50 FPS):

```
CONFIG_INTERVAL_TIMER: @ Asumir reloj de entrada = 100 MHz @ Período deseado
= 20 ms @ Load value =  $100 \text{ MHz} \times 20 \text{ ms} = 2,000,000$  ciclos LDR R0, =TIMER_BASE
LDR R1, =2000000 @ 20 ms a 100 MHz STR R1, [R0] @ Escribir load value LDR
R1, =0x4 @ Habilitar timer + interrupciones STR R1, [R0, #8] @ Escribir control
register
```